Introducción a los Sistemas Operativos de Tiempo Real con FreeRTOS

Arquitectura, Fundamentos y Aplicaciones en Sistemas Embebidos

Universidad: Instituto Tecnológico de Costa Rica

Curso: Taller de Sistemas Embebidos

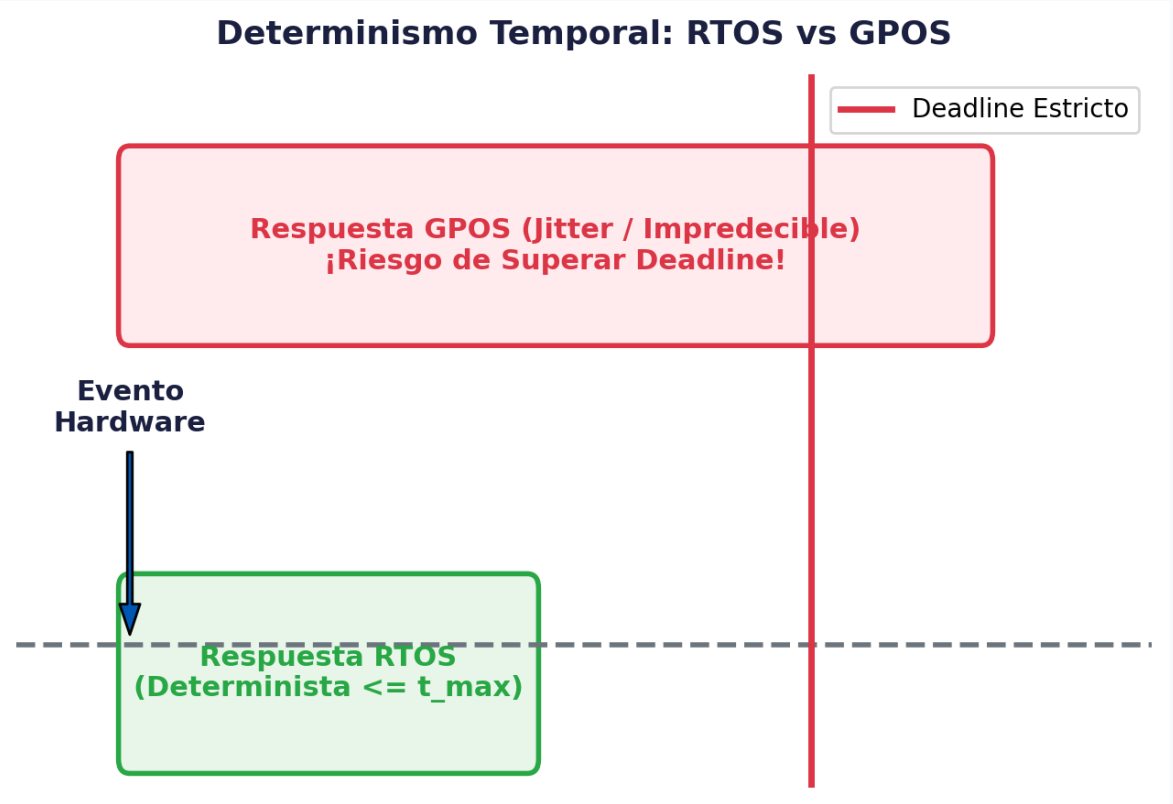
Estudiante: **José David Vargas García**

Fecha: 2 de noviembre del 2026

Enfoque: Sistemas Operativos de Tiempo Real (RTOS) y Kernel FreeRTOS

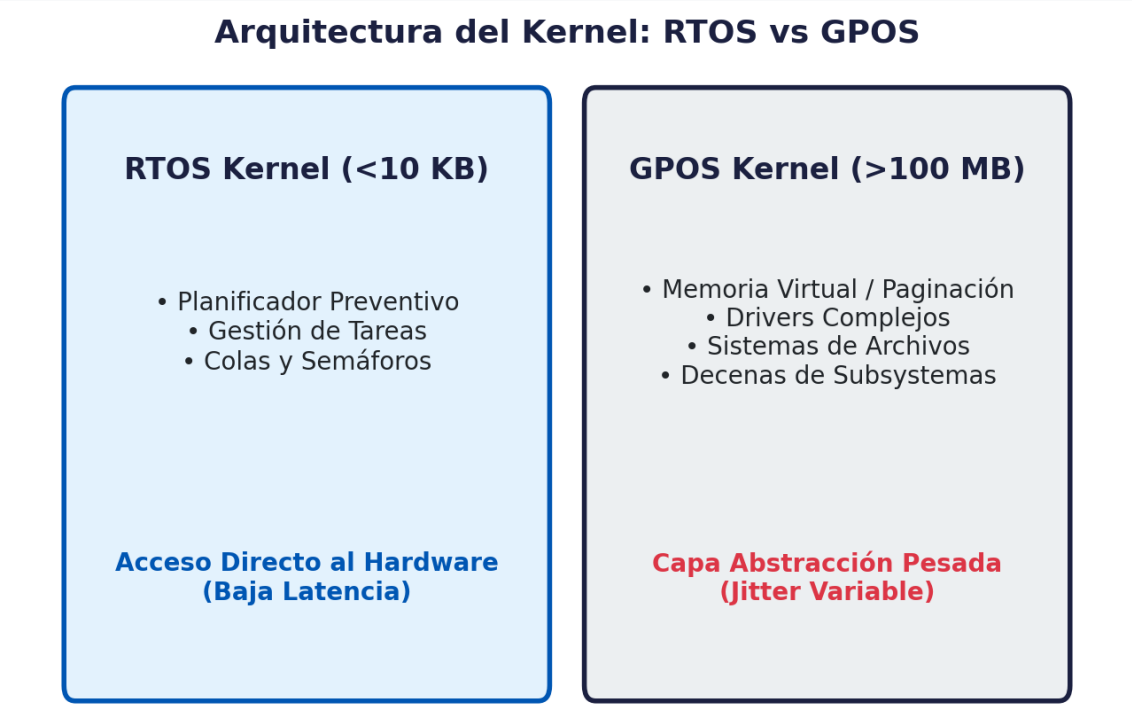
Diapositiva 2: ¿Qué es un Sistema Operativo de Tiempo Real?

- **Definición de RTOS:** Kernel ligero diseñado para responder a eventos del mundo real dentro de plazos temporales estrictos (deadlines).
- **RTOS vs GPOS:** Un GPOS (Linux/Windows) busca maximizar el rendimiento promedio (throughput), mientras que un RTOS garantiza determinismo y latencia acotada.
- **Hard Real-Time:** El incumplimiento de un solo deadline provoca un fallo catastrófico del sistema (frenos ABS, control de vuelo).
- **Soft Real-Time:** El retraso ocasional degrada la calidad del servicio pero no destruye el sistema (transmisión de video, impresión térmica).



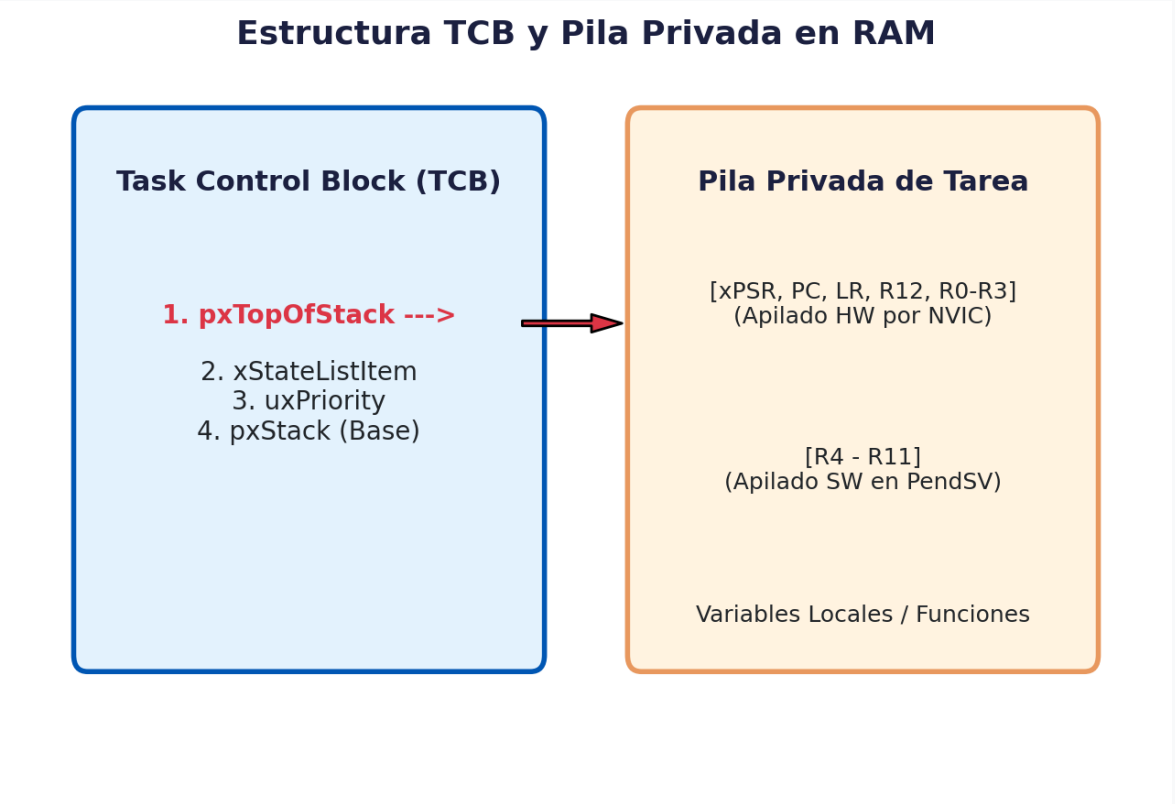
Diapositiva 3: Tabla Comparativa — RTOS vs. GPOS

Criterio	RTOS (FreeRTOS)	GPOS (Linux/Windows)
Objetivo	Determinismo y latencia acotada	Rendimiento promedio (Throughput)
Planificación	Prioridades estrictas preventivas	Reparto equitativo (Time-slicing)
Latencia ISR	Mínima, acotada y predecible	Variable e impredecible
Huella Memoria	512 B - 10 KB Flash; 1.5 - 15 KB RAM	Cientos de Megabytes / Gigabytes
Gestión RAM	Pistas fijas / Asignación estática	Memoria Virtual y Paginación



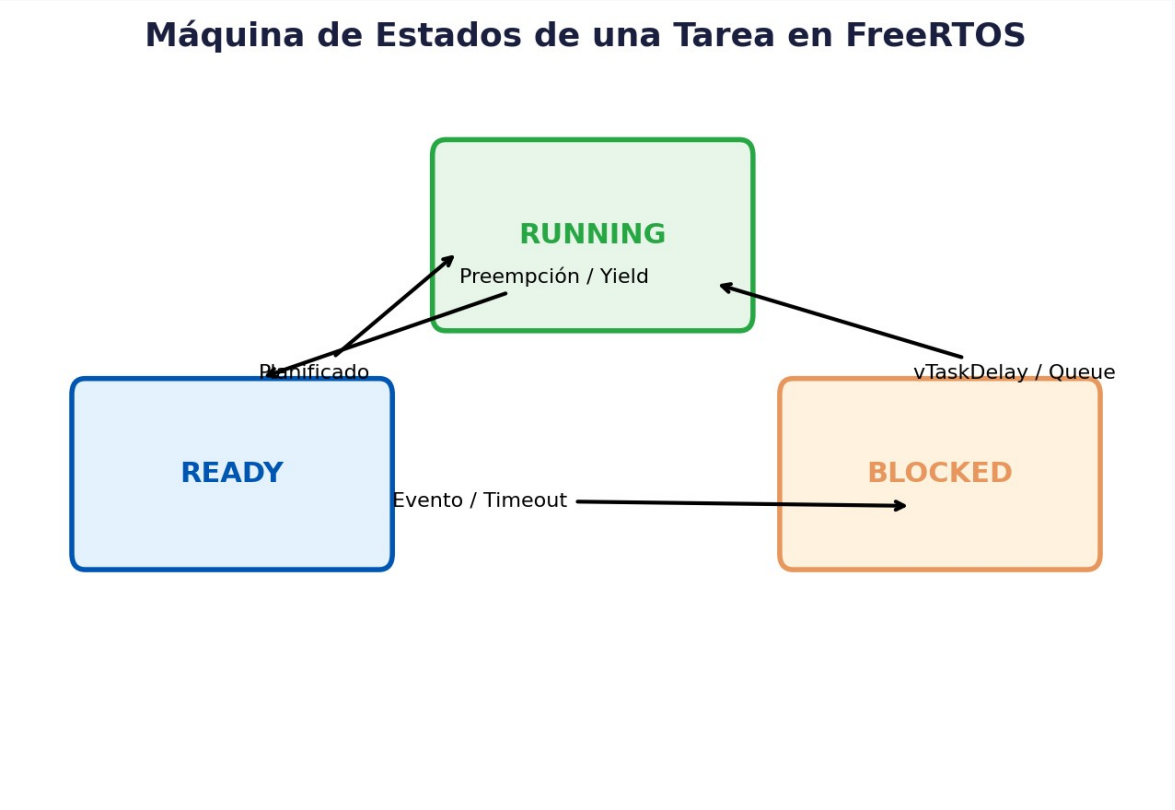
Diapositiva 4: Concepto de Tarea (Task), TCB y Pila Privada

- **Concepto de Tarea:** Hilo de ejecución independiente encapsulado en un bucle infinito en C (for(;;) / while(1)) que nunca debe retornar.
- **Task Control Block (TCB):** Estructura en RAM que actúa como carnet de identidad de la tarea. Almacena puntero de pila, prioridad y estado.
- **Puntero pxTopOfStack:** Primer miembro obligatorio del TCB para acceso ultrarrápido desde ensamblador durante el cambio de contexto.
- **Pila Privada (Stack):** Bloque de RAM donde la tarea guarda variables locales, parámetros y el marco de registros durante interrupciones.



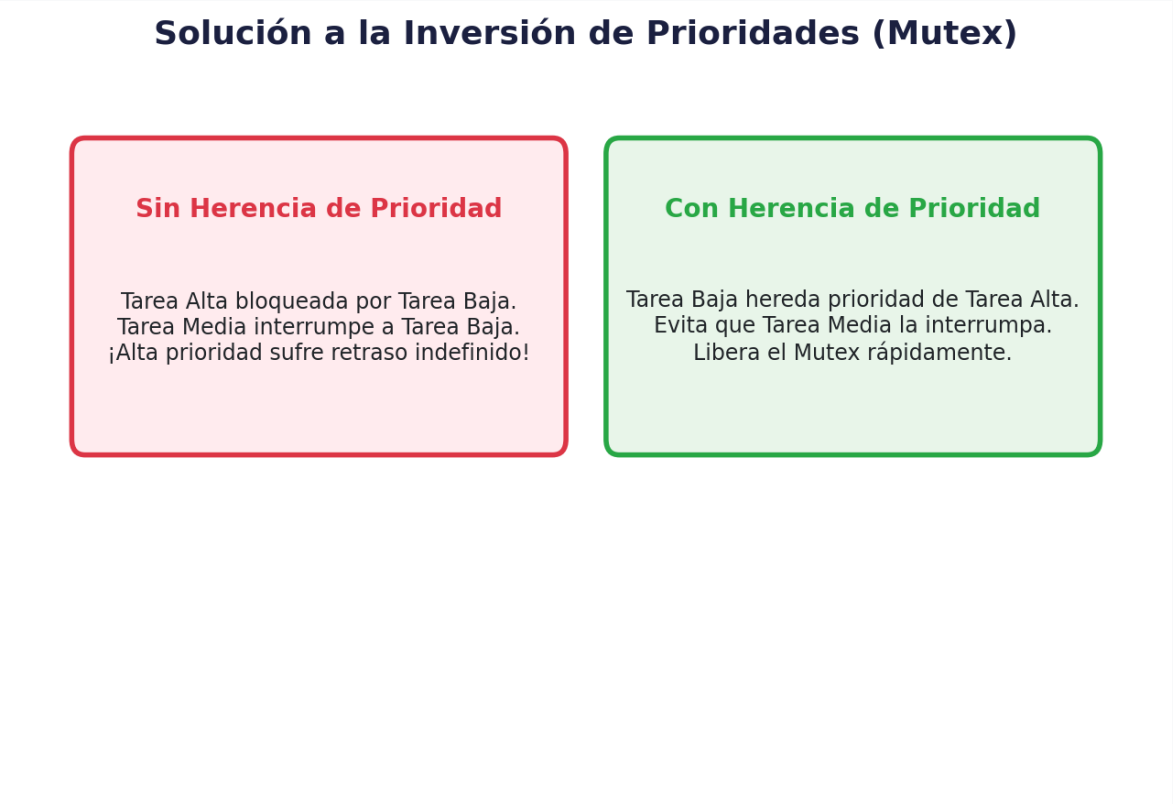
Diapositiva 5: Planificación Preventiva y Estados de Tarea

- **Planificación Preventiva:** El planificador garantiza que la tarea en estado Ready con la máxima prioridad obtenga el control exclusivo de la CPU.
- **Optimización HW CLZ:** En ARM Cortex-M se utiliza la instrucción Count Leading Zeros para seleccionar la tarea lista en tiempo constante O(1).
- **Estado Running:** Tarea ejecutándose actualmente en el núcleo del procesador.
- **Estados Ready / Blocked:** Ready: Lista para ejecutar. Blocked: Esperando un evento externo (sensor, semáforo, delay).



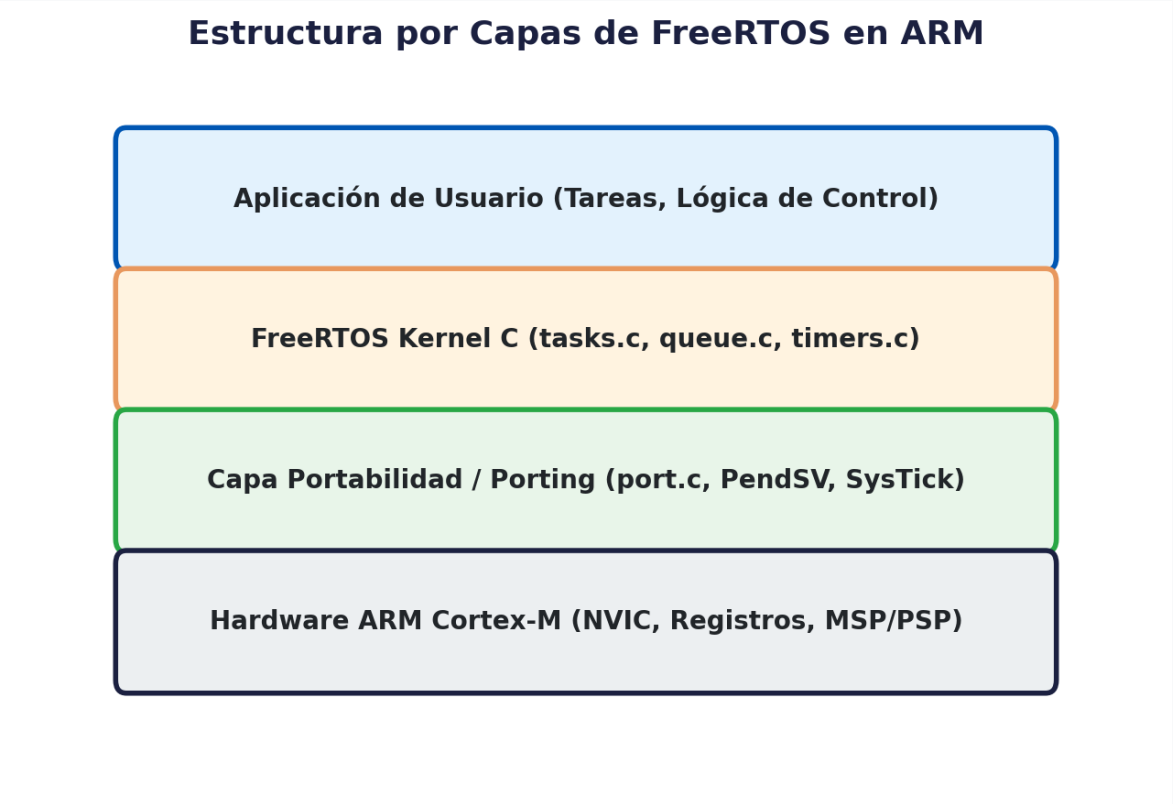
Diapositiva 6: Mecanismos de Sincronización e Inversión

- **Mecanismos IPC:** Colas FIFO seguras para paso de mensajes por copia. Semáforos y Mutexes para exclusión mutua.
- **Inversión de Prioridades:** Ocurre cuando una tarea de alta prioridad queda bloqueada por una de baja que sostiene un recurso, mientras una media impide avanzar.
- **Herencia de Prioridades:** FreeRTOS eleva temporalmente la prioridad de la tarea baja al nivel de la alta hasta que libere el Mutex.
- **Garantía de Tiempo Real:** Evita retrasos indefinidos en tareas críticas del sistema.



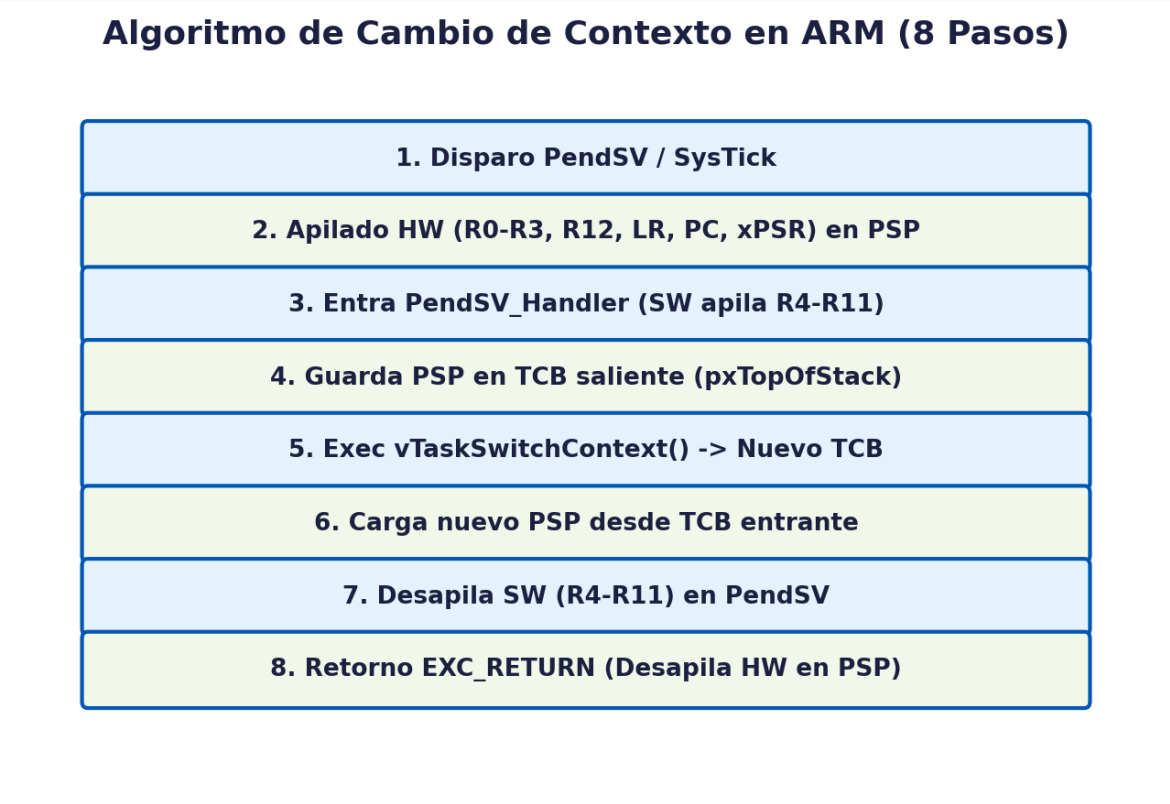
Diapositiva 7: Estructura del Kernel y Capa de Portabilidad

- **Núcleo C Independiente:** Módulos portables en C puro: tasks.c, queue.c, timers.c, list.c.
- **Capa de Portabilidad:** Archivos específicos para el procesador: port.c, portmacro.h, portasm.s.
- **FreeRTOSConfig.h:** Archivo central de configuración para habilitar/deshabilitar módulos y ajustar la huella de memoria.
- **Doble Puntero de Pila:** MSP (Main Stack Pointer) para interrupciones y kernel; PSP (Process Stack Pointer) para tareas.



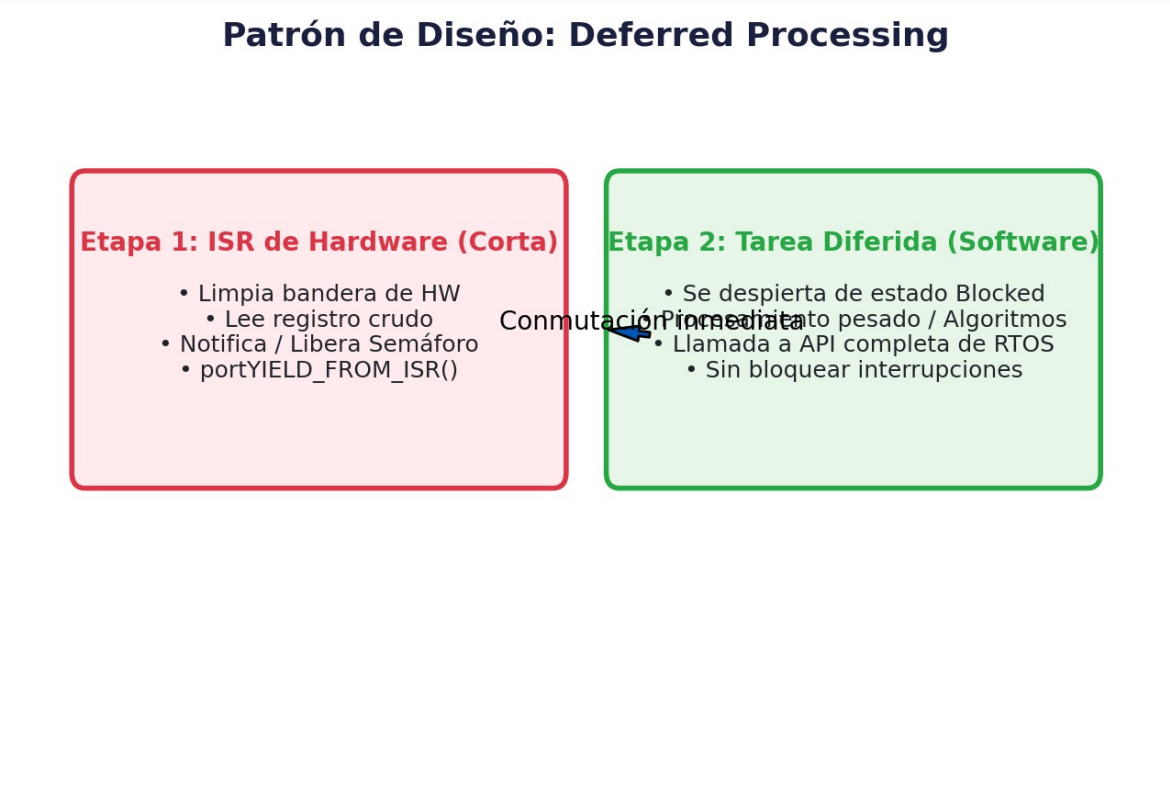
Diapositiva 8: Algoritmo de Cambio de Contexto en ARM

- **Disparo del Context Switch:** Invocado por el temporizador SysTick o la función API portYIELD() marcando el bit de PendSV.
- **Apilamiento HW (Caller-saved):** El hardware de ARM guarda automáticamente R0-R3, R12, LR, PC y xPSR en la pila PSP.
- **Apilamiento SW (Callee-saved):** PendSV_Handler guarda R4-R11 en PSP y actualiza pxTopOfStack en el TCB saliente.
- **Conmutación y Desapilamiento:** vTaskSwitchContext() selecciona la nueva tarea; se restauran R4-R11 y el hardware desapila R0-R3.



Diapositiva 9: Explicación Didáctica: Deferred Processing

- **Regla de Oro en RTOS:** Las Rutinas de Servicio de Interrupción (ISRs) deben ser lo más breves y rápidas posible.
- **Etapa 1 (ISR de HW):** Atiende únicamente lo urgente del hardware (limpia banderas, lee registros) y notifica a una tarea.
- **Etapa 2 (Tarea Diferida):** Se ejecuta fuera de la ISR con la prioridad asignada, procesando datos complejos y usando la API completa.
- **Mecanismos:** Implementado mediante xSemaphoreGiveFromISR(), vTaskNotifyGiveFromISR() y portYIELD_FROM_ISR().



Diapositiva 10: Aplicación 1 — Edge AI & TinyML

- **Inferencia en el Borde:** Ejecución de redes neuronales convolucionales (CNN) cuantizadas en microcontroladores (STM32H7 / Cortex-M7).
- **Integración con Frameworks:** Soporte para TensorFlow Lite for Microcontrollers y librerías optimizadas CMSIS-NN.
- **Determinismo y Latencia:** Asignación estática de memoria pools garantizando tiempos de inferencia fijos (ej. 13.8 ms).
- **Eficiencia Energética:** Uso del modo Tickless Idle de FreeRTOS para reducir consumo entre ráfagas de inferencia.



Diapositiva 11: Aplicación 2 — Control Digital de Motores

- **Sistemas de Potencia Digital:** Control de servodrives, robótica e inversores solares con microcontroladores TI C2000Ware.
- **Lazos de Control Rápidos:** Coordinación de algoritmos FCL (Fast Current Loop) y FOC (Field Oriented Control).
- **Buses Industriales:** Comunicaciones en tiempo real con buses CAN, CAN-FD, EtherCAT e interfaz FSI.
- **Seguridad Crítica:** Pre-certificación SAFERTOS bajo normativas IEC 61508 SIL 3 e ISO 26262 ASIL D.



Diapositiva 12: Aplicación 3 — Impresoras Térmicas Industriales

- **Entornos de Alta Carga:** Impresoras industriales de etiquetas en logística, manufactura y puntos de venta.
- **Prevención de Fallos:** Eliminación de desbordamientos de buffer y problemas de sincronización en papel y cabezal.
- **Multitarea Coordinada:** Gestión simultánea de motores de paso, elementos calefactores y buffers USB/Ethernet.
- **Beneficio Operativo:** Reducción de latencias de impresión y disminución de paradas no programadas.



Diapositiva 13: Conclusiones y Preguntas

- **Mínima Huella de Memoria:** FreeRTOS requiere menos de 10 KB Flash y 1.5 - 15 KB RAM, adaptándose a microcontroladores restringidos.
- **Determinismo y Predecibilidad:** Garantiza conmutación de tareas O(1) y respuesta en sub-microsegundos ante eventos críticos de hardware.
- **Arquitectura Robusta y Portabilidad:** Capa de portabilidad limpia que separa la lógica de aplicación del código específico de ARM Cortex-M.
- **Ecosistema Industrial:** Ampliamente adoptado desde nodos Edge AI/IoT hasta sistemas de seguridad crítica con SAFERTOS.

Preguntas y Respuestas

¡Muchas gracias por su atención! Espacio abierto para dudas y comentarios de la audiencia.

Diapositiva 14: Referencias Bibliográficas

📚 Libros y Fundamentos Teóricos

• **Tanenbaum, A. S. (2009).** Sistemas Operativos Modernos (3ª ed.). Pearson Educación.

• **Siewert, S., & Pratt, J. (2015).** Real-Time Embedded Components and Systems with Linux and RTOS (2nd ed.). Mercury Learning.

• **Valdés Pérez, F. E., & Pallás Areny, R. (2007).** Sistemas de Tiempo Real: Introducción al Diseño. Marcombo.

• **UNICEN. (2004).** Una Panorámica del Problema de Inversión de Prioridades en Sistemas Operativos de Tiempo Real. UNICEN.

🛠️ Documentación Oficial y Manuales

• **Real Time Engineers Ltd. / AWS. (2017).** Mastering the FreeRTOS™ Real Time Kernel: A Hands-On Tutorial Guide. Amazon Web Services.

• **Real Time Engineers Ltd. / AWS. (2018).** The FreeRTOS™ Reference Manual: API & Configuration Guide (v10.0.0). Amazon Web Services.

• **FreeRTOS.org. (2024).** FreeRTOS Kernel Core, Configuration & Porting Documentation. Amazon Web Services.

• **CosmicLearn. (2022).** RTOS FreeRTOS: Core, Configuration & Porting Mechanics.

Diapositiva 15: Referencias Bibliográficas

⚙ Arquitectura e Implementación

• **Memfault (Interrupt). (2021).** ARM Cortex-M RTOS Context Switching & PendSV Exception Handler Mechanics.

• **Texas Instruments Incorporated. (2020).** C2000™ Software Guide (Rev. v1.5) & SimpleLink™ MCU Platform (Rev. E).

• **WITTENSTEIN High Integrity Systems.** Functional Safety Standards for SAFERTOS® (IEC 61508 SIL 3 / ISO 26262 ASIL D).

🏭 Aplicaciones Industriales y Edge AI

• **IJTERD. (2023).** Real-time Operating Systems (RTOS) For Edge AI & TinyML Benchmarking in Microcontrollers.

• **Journal of Quantum Science and Technology. (2022).** Optimizing Thermal Printer Performance with On-Time RTOS for Industrial Applications.

• **Valdés, F., et al.** Electronics for Embedded Systems & Passive Circuit Analysis for Real-Time Controllers.